Reproducible projects in R:

practical workshop, part 2

Dr Marina Vabistsevits
Research Fellow in Clinical and Biomedical Sciences
UK Reproducibility Network (UKRN) Local Network Lead for Exeter

What this workshop is about

Part 1: (last week)

Project-oriented workflow:

- Organising your projects with .Rproj
- Using renv to manage packages in your Rstudio environment
- Reproducibility habits and efficiency tips

Part 2: (today!)

Using Git to track changes in your R project

Icon for R

Icon for RStudio

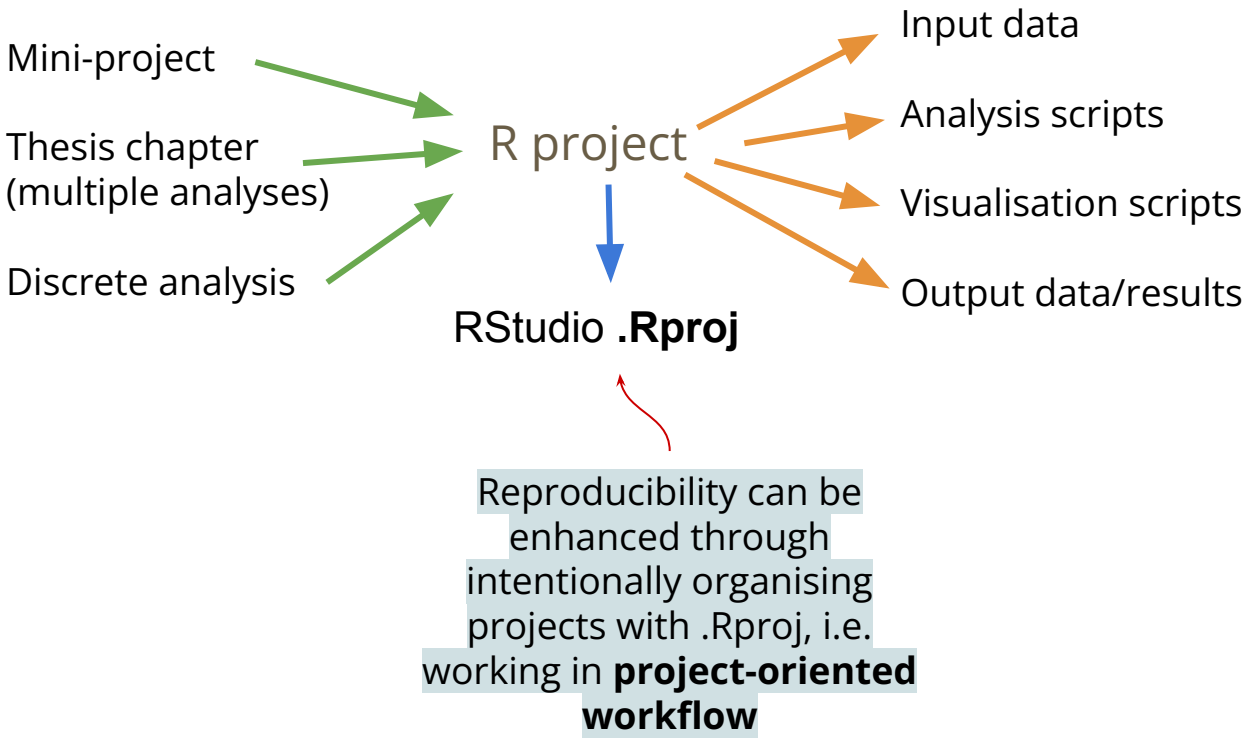
software company who develop Rstudio

Positron - new IDE from Posit for R/Python etc

The slides will be shared after!

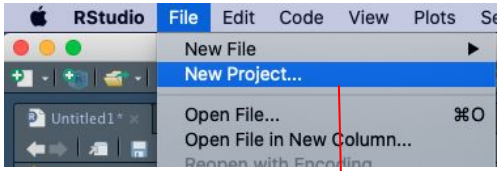
Recap: using R projects and R env

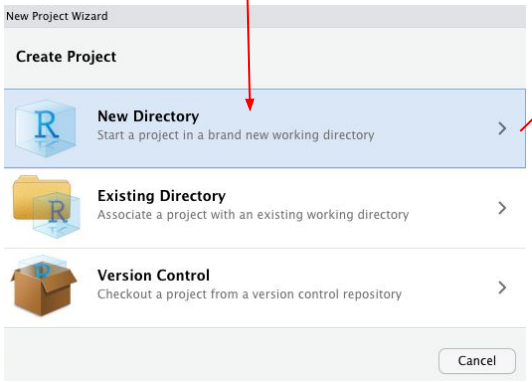
R projects

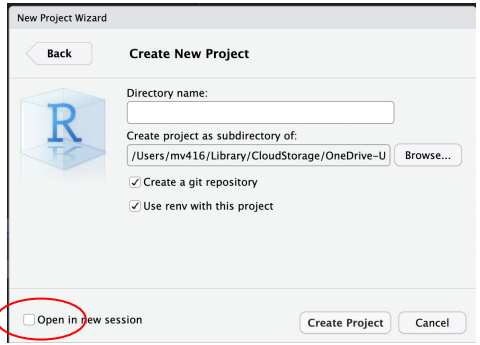

Mini-project
Thesis chapter
(multiple analyses)
Discrete analysis
R project
Input data
Analysis scripts
Visualisation scripts
Output data/results
RStudio .Rproj
Reproducibility can be enhanced through intentionally organising projects with .Rproj, i.e. working in **project-oriented workflow**

Project-oriented workflow

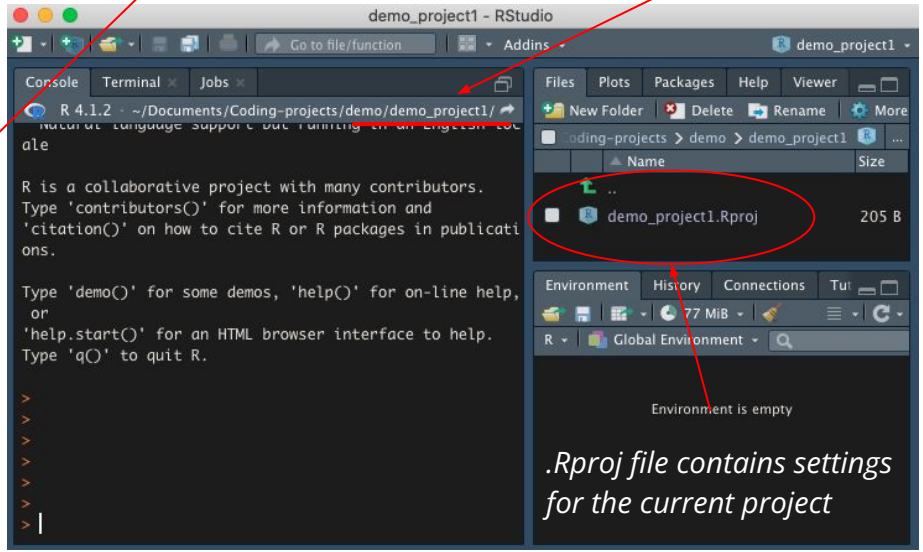
Creating your new project in Rstudio as .Rproj:







Project directory



This means that you are essentially compartmentalizing your current project

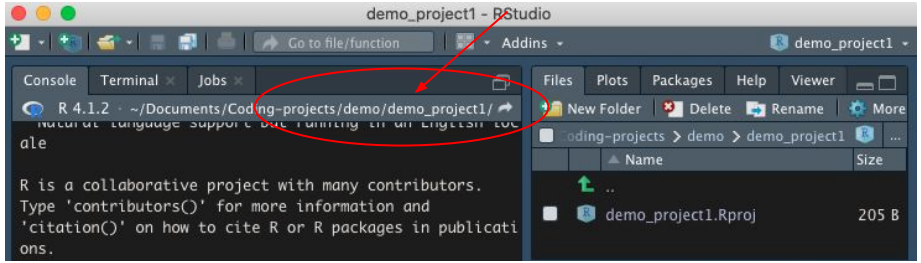
How working within .Rproj improves reproducibility:

File system discipline: project directory stores all your data, scripts, figures

File path discipline: The working directory is set to the project directory (e.g. **demo_project1/**), so you don't need to *setwd()* or specify full paths to data (only internal subfolders that are relative to top directory)

Working directory intentionality: when working on project A, your working directory is set to project A's folder

Project working directory

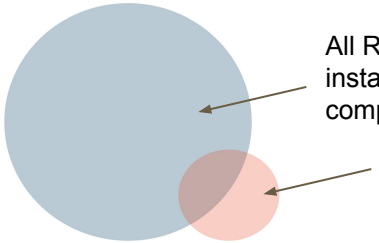


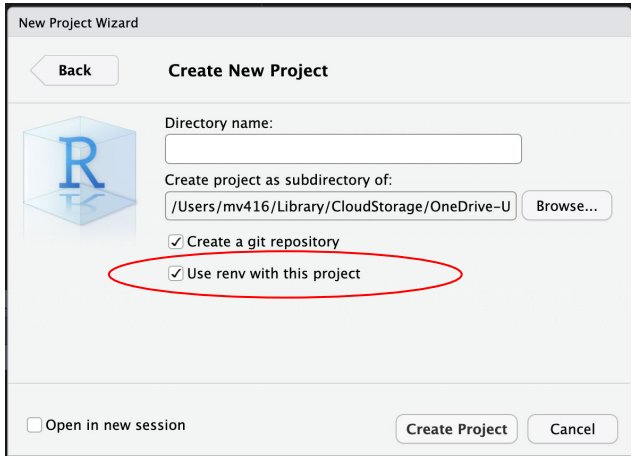
- The project **creates everything it needs**, within its workspace/folder, and **touches nothing it did not create**
- Any scripts are written assuming they will be **run from a fresh R session** within the project
- The project folder **can be moved *anywhere*,** and everything will still work (no paths will be broken)

renv for managing package versions within projects

renv package helps you create reproducible environments for your R projects

- Keeps track of all your installed packages
- Installs them in a new location (i.e. easy to share)
- Allows to come back to an old project without code breaking due to updates in package versions





Setting up your renv

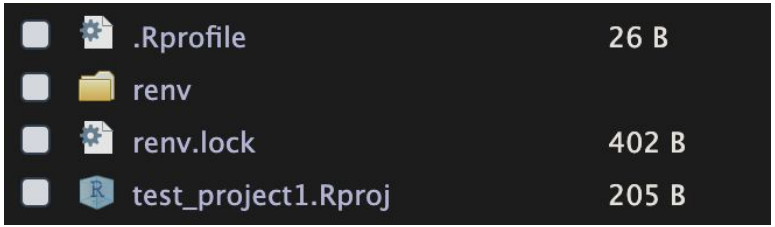
`install.packages("renv")`

`renv::init()` – initialise your project env

NB this is the same as ☑ Use renv with this project

This creates:

- **renv/**: stores packages for the project.
- **renv.lock:** records packages and the exact versions used
- **.Rprofile**: ensures renv activates when the project opens



When renv is initialised, your project gets its own project library - a private folder (**renv/library/**) where packages are installed just for this project.

This means package versions are isolated and recorded for each project, so updating a package in one project won't break others.

This makes collaboration and reproducibility much easier.

`renv::snapshot()` – update your env with newly installed packages (saves to renv.lock)

`renv::restore()` – when project is re-opened, load all your required packages

Using Git to track changes in your R projects

https://happygitwithr.com/

Checklist before we start

- GitHub account
- Personal access token
- Git is available on your laptop
- SSH-key (linking Git on your laptop with your GitHub account)

Git brief intro

Git - software on your computer that records your project history

Repository - a folder on your computer that Git is watching

Github - a website that hosts you repository online for sharing, back-up, and collaboration

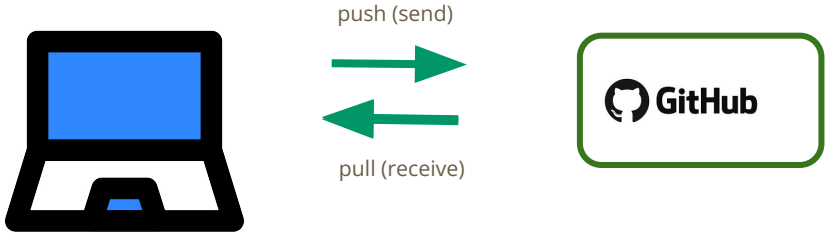
Why use it?

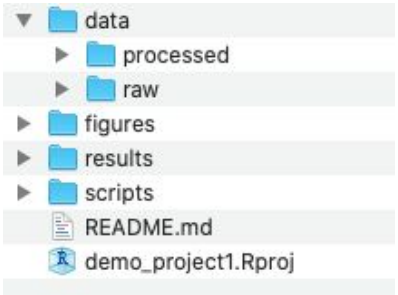
- Improves reproducibility (e.g. keeps record of changes)
- Enables code sharing with colleagues / as a part of publication
- Can facilitate data/code integrity
- Important skill for anyone working with data (e.g. can showcase your work during job applications)

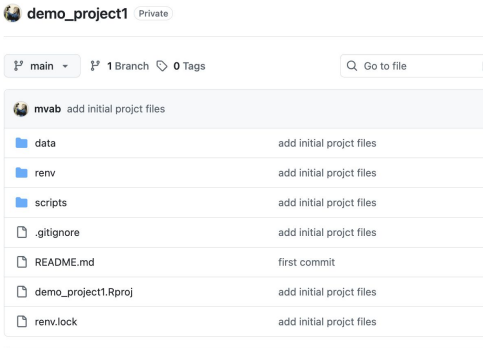
Why might you want to use git?

Your R project on local PC
Local repository

Your R project as GitHub repo
Remote repository

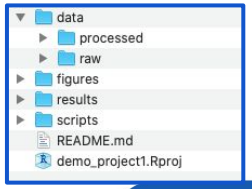






- Using git helps you keep track of your daily changes
- All your work is backed-up (as long as you do it regularly and intentionally)
- “Collaborating with yourself” in one place

Basic git commands / actions



Analysis folder on your local computer

Git local repo will keep a record of the new changes that you want save

Local

Remote

working directory

staging area

local repo

remote repo

Your analysis folder repo on Github remote repo with all changes tracked, with a commit message that explains what has changes

git add

Tell Git which files you changed/created, and want to track with Git

git commit

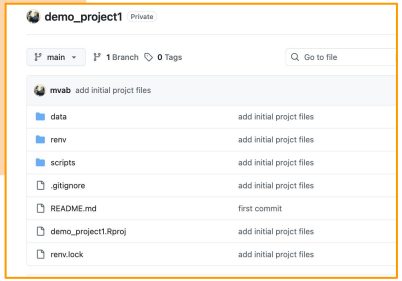
Send your updates to the remote repo on Github

git push

git pull

Tell Git what your new changes mean/do via a commit message

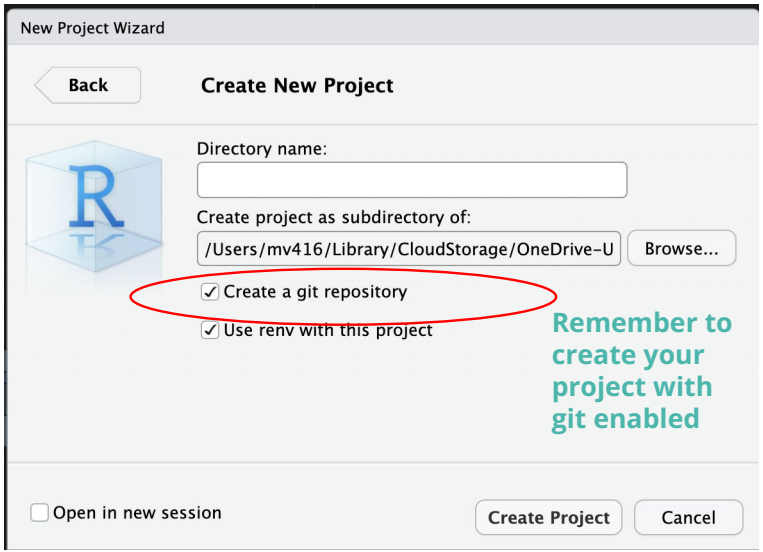
git checkout



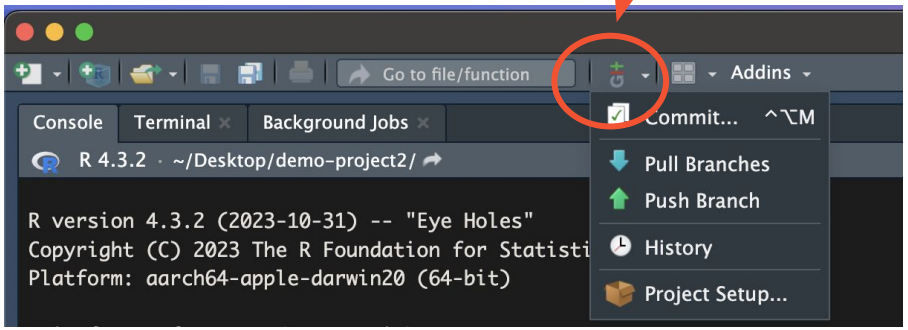
Follow along if you can

Using git with your .Rproj

- Rstudio user interface makes it easy to get started with git
- (but also can use the terminal to run git commands)



This means Git is enabled in the current project (use drop-down to interact with it)



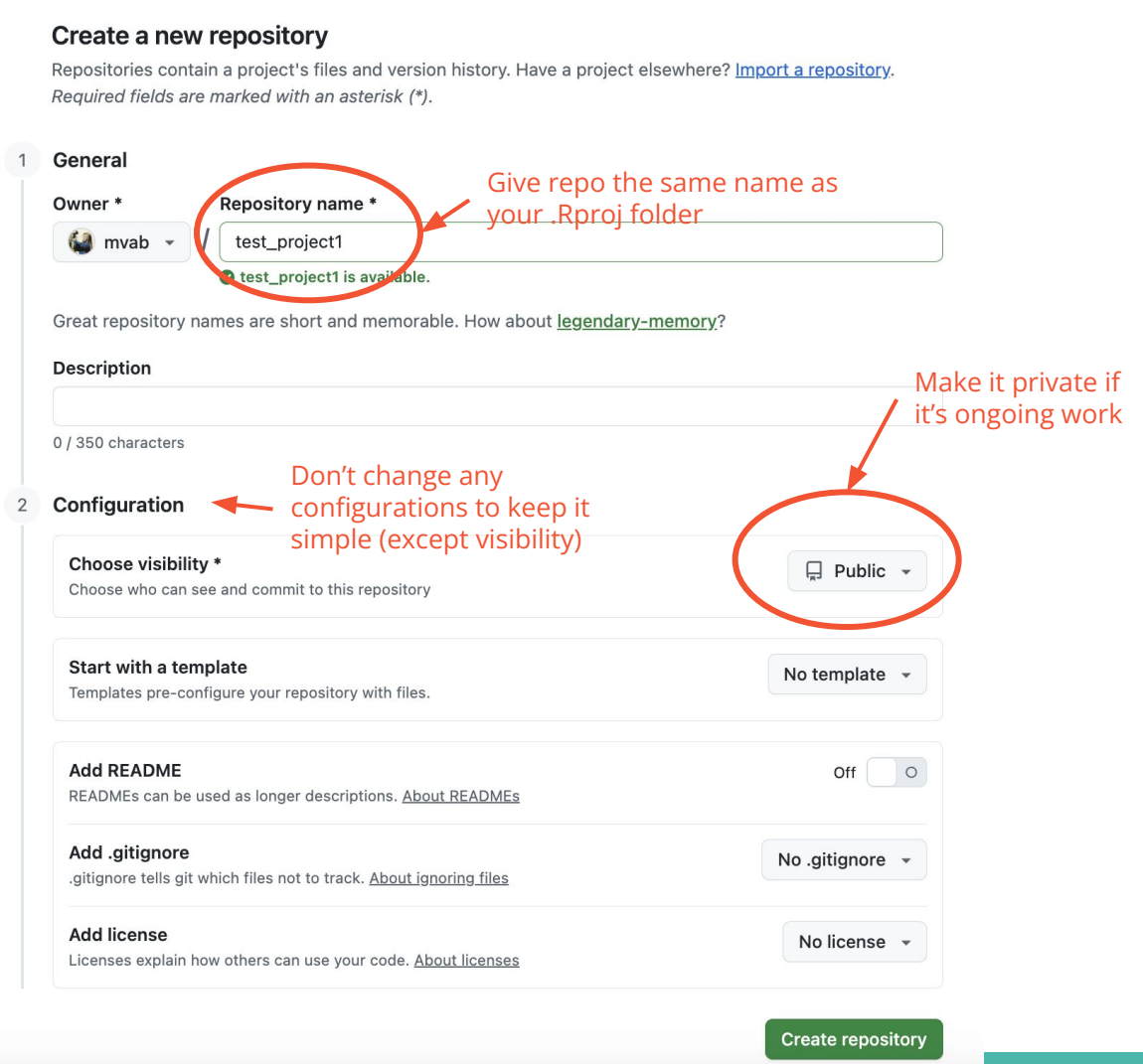
-> Next need to link your local project folder to a GitHub repo

Create remote repo

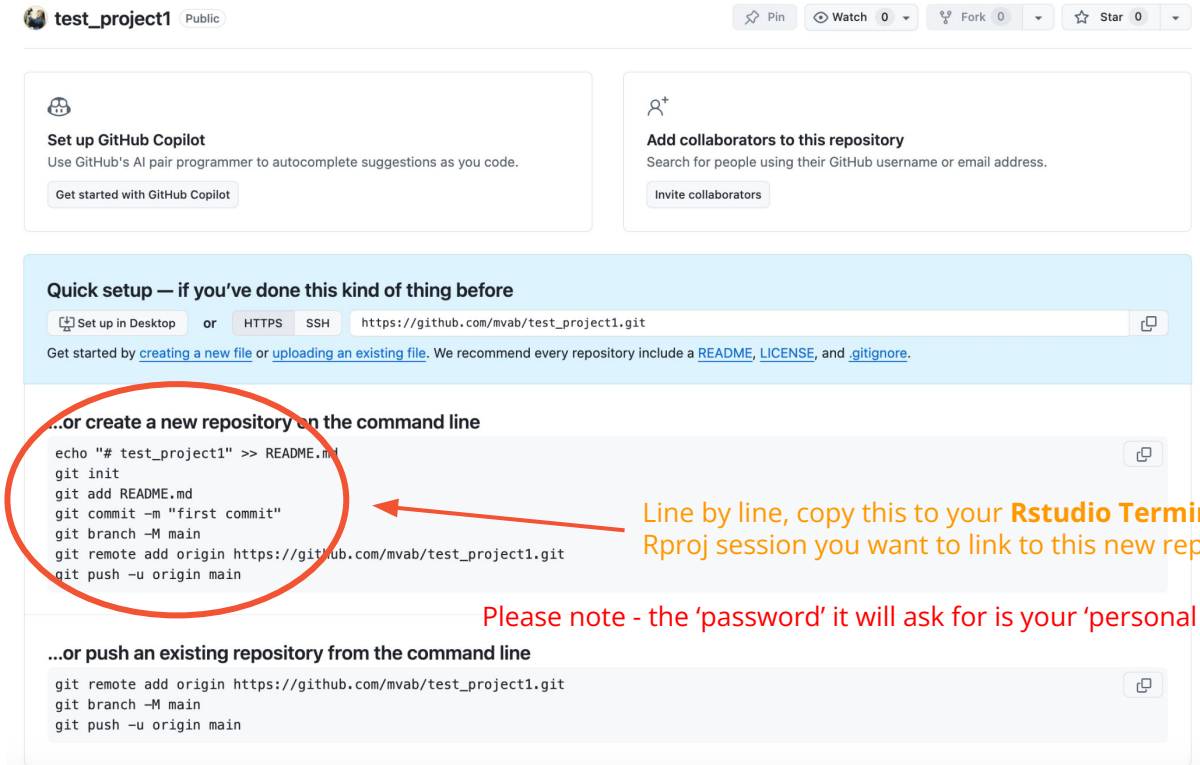
After logging in your Github account:

To create a new repository:

Repositories -> New



GitHub makes it easy to link your .Rproj to the new repo:



Line by line, copy this to your **Rstudio Terminal** in the Rproj session you want to link to this new repo

Please note - the 'password' it will ask for is your 'personal access token'

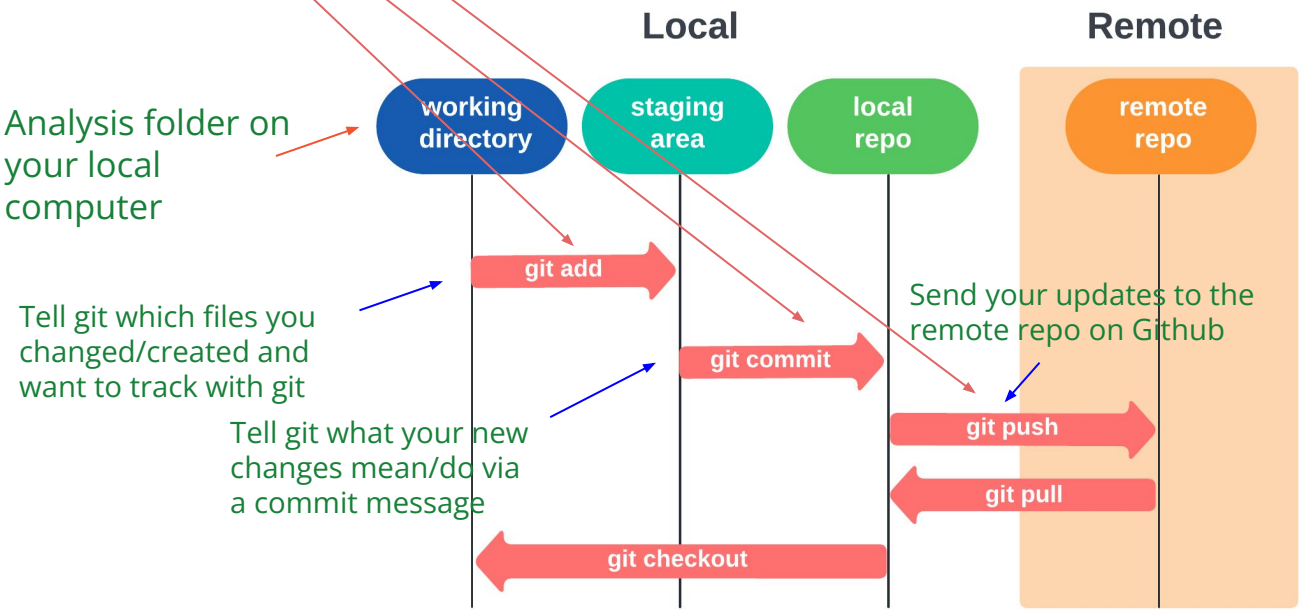
Only need to do this once!

```
echo "# test_project1" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/mvab/test_project1.git
git push -u origin main
```

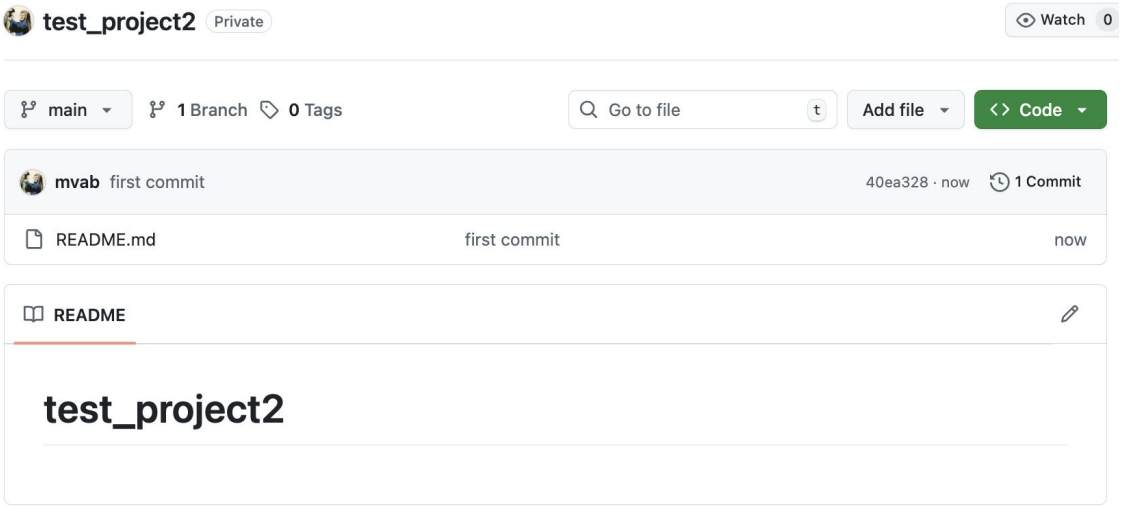
<- creating README.md file with ".." text in it - can change it!

<- initialises your project repo

initialises your project repo



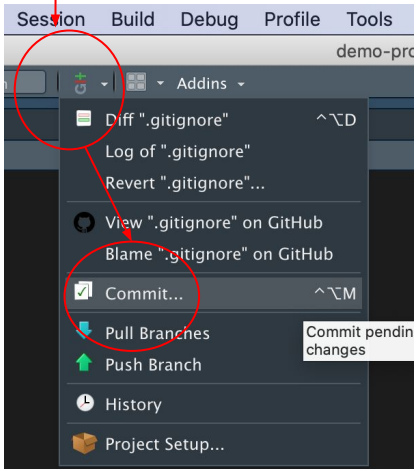
Remote repo after first commit



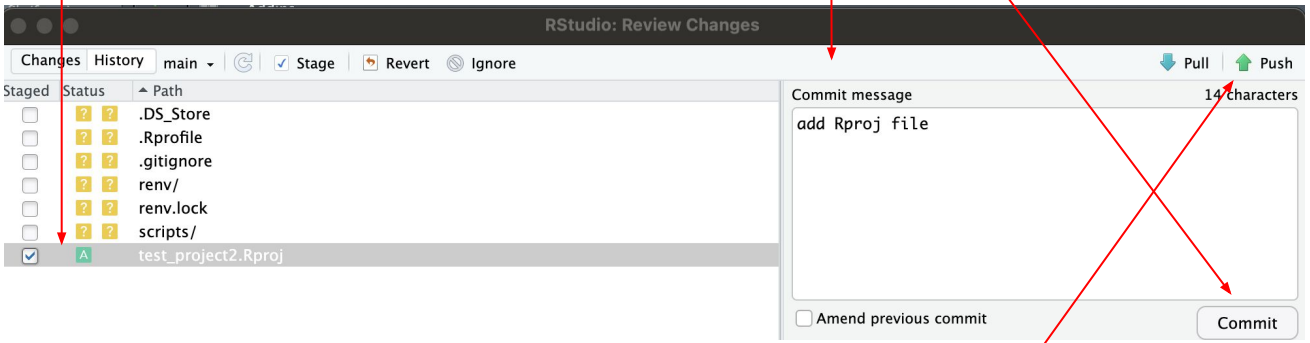
Only README file has been added

Time to add other files!

1) Git button - > Commit



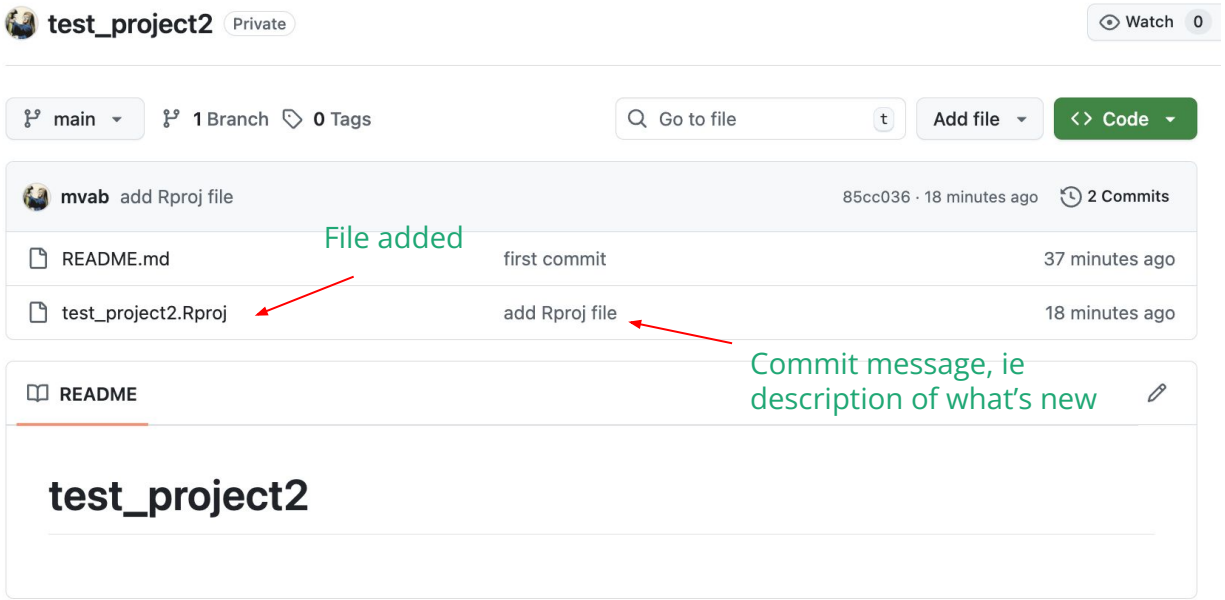
2) Select the files you want Git to **add** (track):



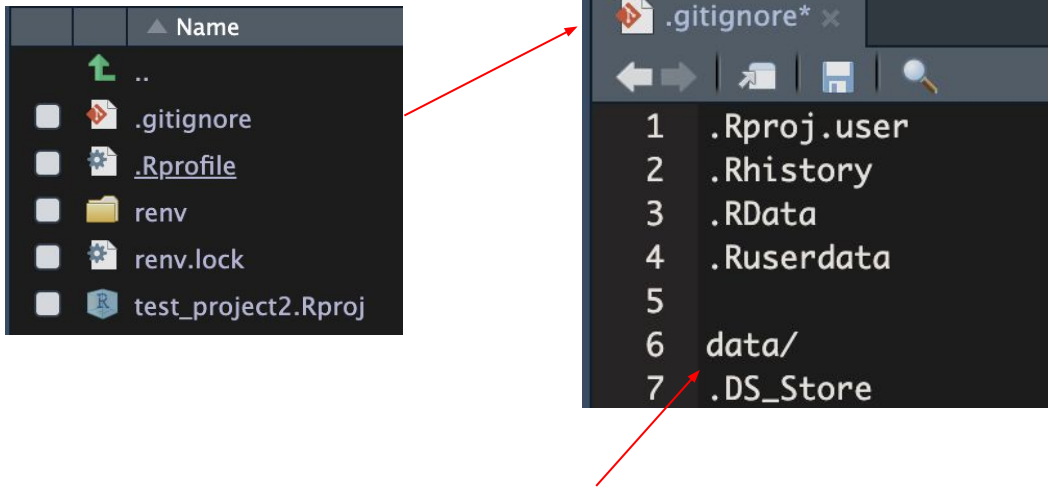
3) write your 'commit message', ie description of your change (tip: write it as an instruction), then press "**Commit**"

4) Press "**Push**" to send the change to remote repo

Your change has been added:



.gitignore file - list files you don't want to store in remote repo



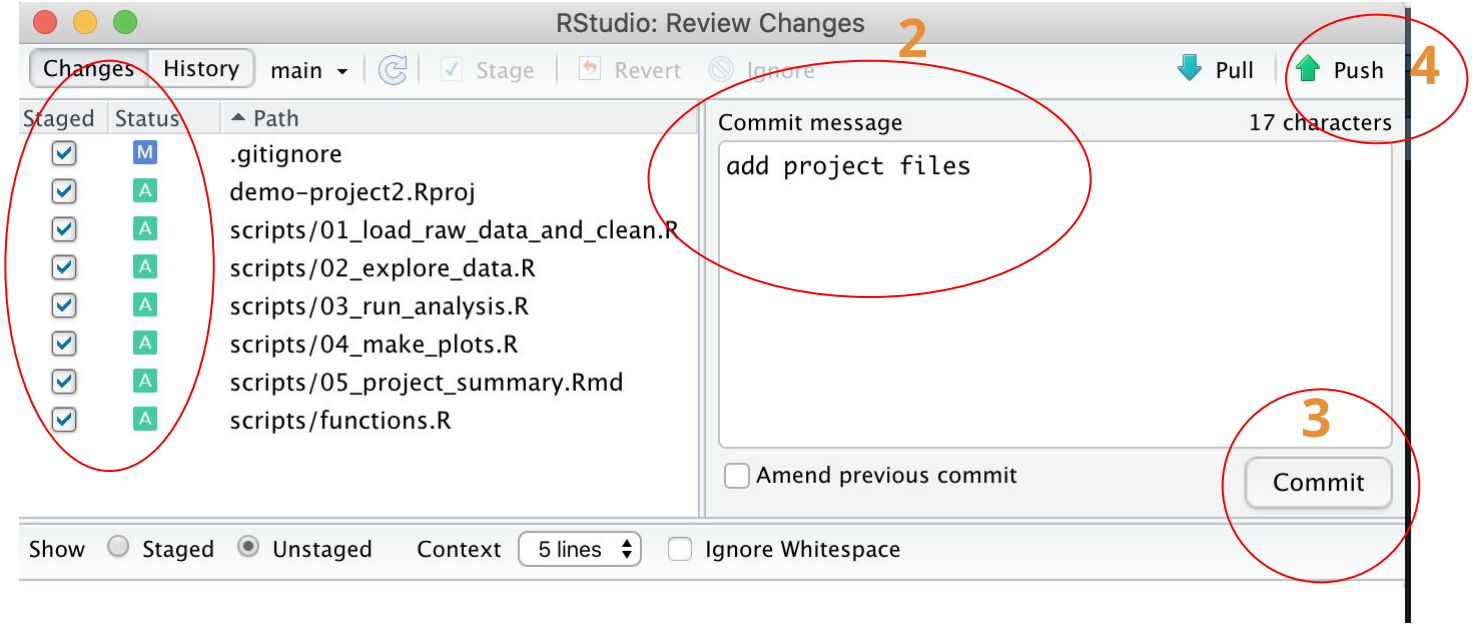
Add **data/** folder to *.gitignore* file so that your data files (if large or sensitive) are not committed to your project repo on Github

- list files and folders in your project folder in the *.gitignore* file - they will be ignore by Git (this prevents them from being accidentally added to GitHub repo)
- save and '**add**' file
- **commit** and **push** the updated *.gitignore* file to your repo

Now add more files:

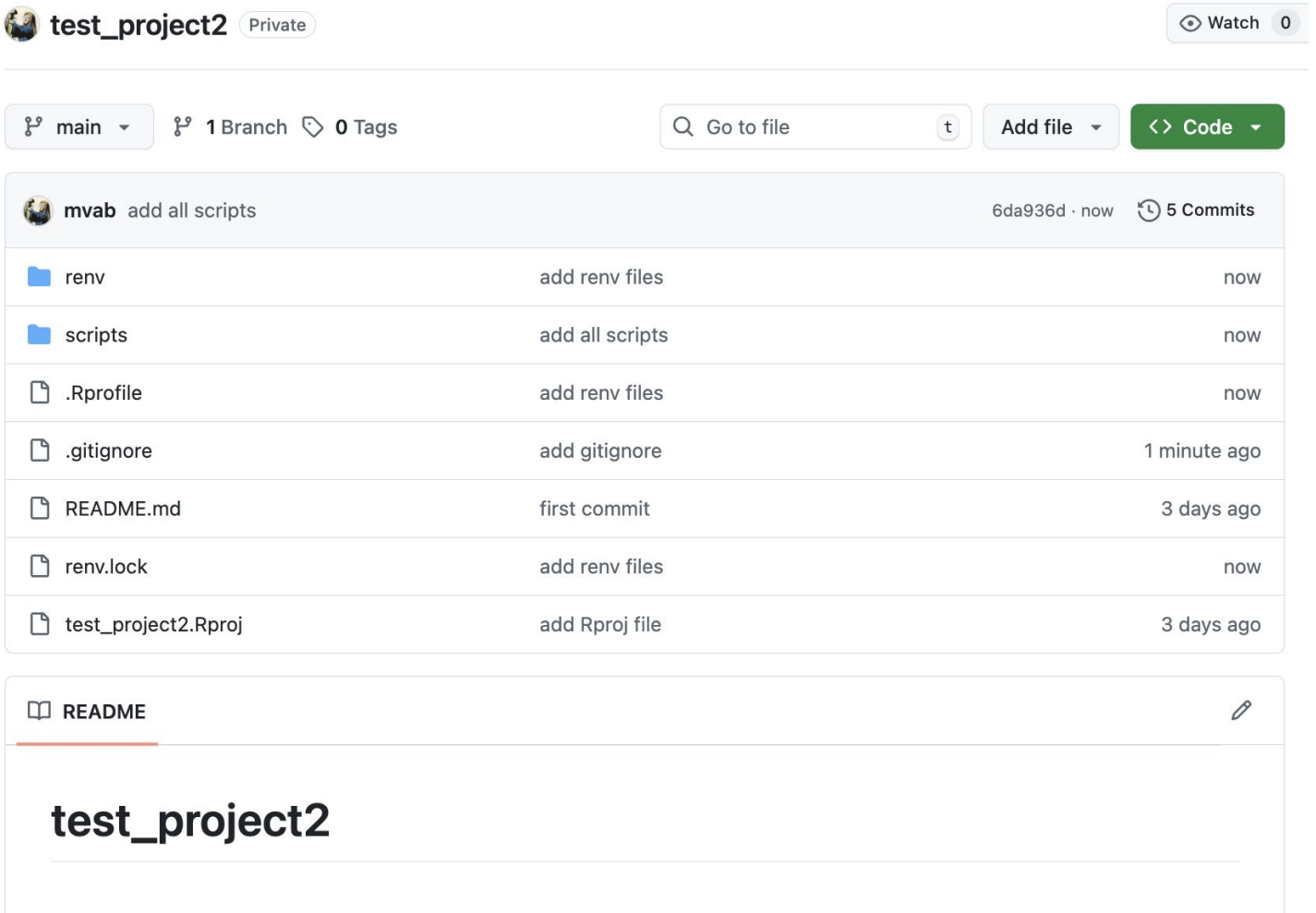
Commit changes:

Add message:

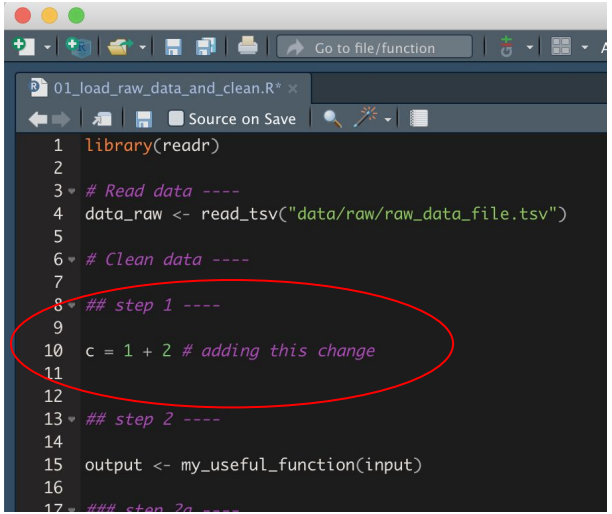


Get files: https://tinyurl.com/4czpk3cb

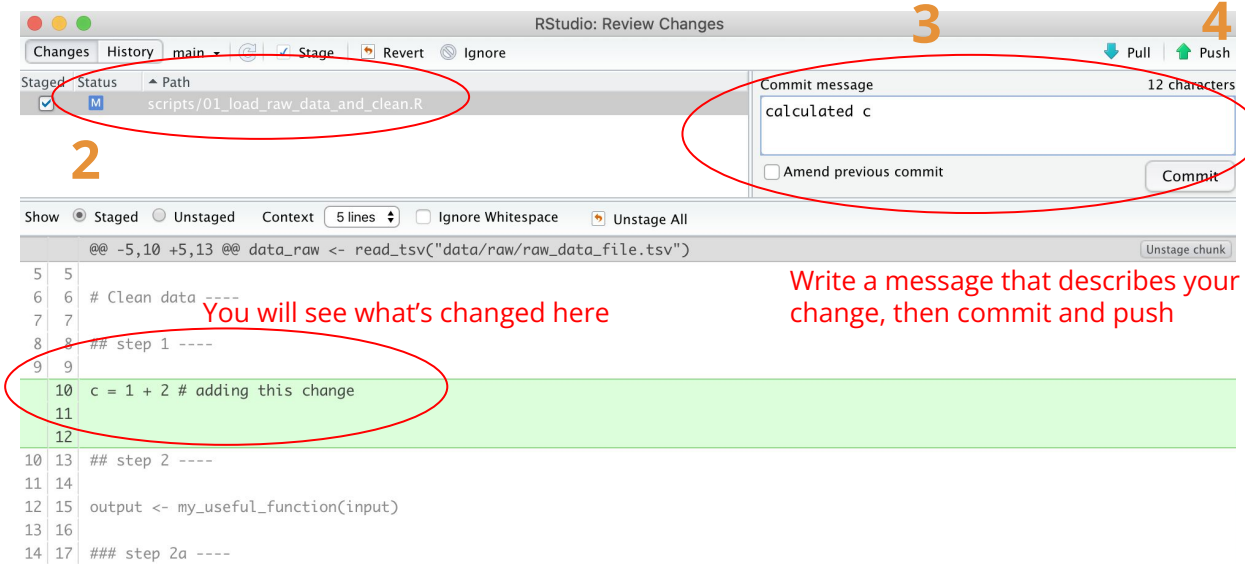
Your changes on Github:

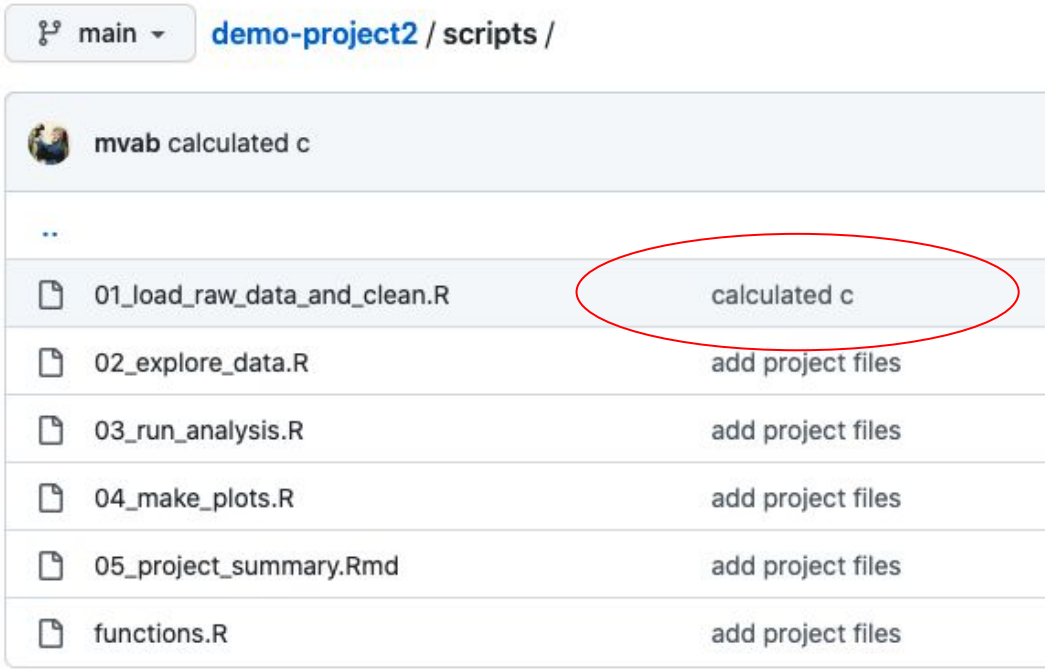


Practise adding a specific change to a script:



1




main
demo-project2 / scripts /
mvab calculated c
..
01_load_raw_data_and_clean.R
calculated c
02_explore_data.R
add project files
03_run_analysis.R
add project files
04_make_plots.R
add project files
05_project_summary.Rmd
add project files
functions.R
add project files

Good practice principles

1. **Add** and **commit** your changes often and in small chunks
    - E.g. separate commits for different files and sections in your analysis steps
2. Write descriptive commit messages of what you've changed

Good
- Remove duplicate patient encounters from EHR extract
- Handle missing values using median imputation
- Engineer Comorbidity Index features
- Train XGBoost model for sepsis prediction

Bad
- fix
- update
- WIP
- final

3. Use .gitignore
4. Keep a project README
    - describes what your scripts do, and where external data is stored

Summary: using .Rproj for organising work

- "Reproducible project in R" means:
  - **File system discipline:** all files related to a single project are stored in a designated folder;
  - **Working directory discipline**: intentionally work in project directory when opening Rproj
  - **File path discipline:** paths are relative to the project directory (not hard-coded full paths!)
  - **Daily work habit:** Restarting R often and re-running your script under development from the top will help you catch issues early on
  - **Using git for version control**

- Practising these habits together will give you the biggest pay-off
  - Reproducing your analyses will be easy
  - Organising your projects will help you make sense of them in 6/12/etc months
  - Can move your project anywhere or share it with anyone without changing paths

https://rstats.wtf/project-oriented-workflow.html#work-in-a-project

Recommended and used resources

https://www.tidyverse.org/blog/2017/12/workflow-vs-script/

https://richpauloo.github.io/2018-10-17-How-to-keep-your-R-projects-organized/

https://www.rforecology.com/post/organizing-your-r-studio-projects/

https://kkulma.github.io/2018-03-18-Prime-Hints-for-Running-a-data-project-in-R/

https://rstats.wtf/project-oriented-workflow.html

https://appsilon.com/rstudio-shortcuts-and-tips/

https://datacornering.com/my-favorite-rstudio-tips-and-tricks/

https://happygitwithr.com/

https://rstudio.github.io/renv/articles/renv.html

https://rstats.wtf/

https://youtu.be/fTVp7K38jgQ?si=lLLFHMV3t3qKT7sa

Thank you!

Promoting Open and Reproducible Research at Exeter

University of Exeter UK Reproducibility Network (UKRN)

The University of Exeter is a member of UK Reproducibility Network (UKRN).

UKRN functions as a nationwide consortium driven by peers, with the overarching goal of maintaining the UK's position as a hub for cutting-edge research. This objective is achieved through an exploration of the elements that bolster resilient research, the facilitation of training initiatives, and the widespread sharing of exemplary methodologies. Through collaborative partnerships with external stakeholders, the UKRN ensures seamless alignment of endeavors within the sector.

JOIN THE UKRN EXETER MAILING LIST

Stay informed on upcoming event and initiatives

https://www.exeter.ac.uk/research/openresearch/reproducibility/ukrn/